

자연들에 — LLM 처리 상세 프로세스 (Top-Down)

Top-Down 상세 프로세스 | 기준: agri_ai_core/ | 작성일: 2026-04-18

L1. 클라이언트 요청 접수 — FastAPI 엔드포인트

#	디렉토리	파일명	함수명	함수기능역할
1	api/	app.py	query_llm / query_llm_stream	REST API 엔트리. /api/v1/query(동기) 및 /api/v1/query/stream(SSE) 수신 → verify_api_key 인증 → QueryRequest Pydantic 검증 후 L2로 위임.
2	api/	app.py	JsonLoggingMiddleware.dispatch	요청 본문/응답 본문을 마스킹·JSON 포맷으로 로깅 (API 관측용).
3	api/	models.py	QueryRequest / QueryResponse	멀티턴 session_id, user_query, file_paths, farm_id, house_id, speech_style 등 입력/출력 스키마 정의.

L2.질의 핸들러 진입 (동기·스트리밍 통합)

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	query_handler_simple.py	query_llm_simple	동기(JSON) 응답 진입점. 세션 로드 → 파일 첨부 처리 → 3단계 파이프라인 비동기 실행 → 결과 반환
2	src/ai/	query_handler_simple.py	query_llm_simple_stream	SSE 스트리밍 진입점. ThreadQueue를 통해 단계별 진행도를 실시간 송출.
3	src/ai/	query_handler_simple.py	_build_default_tool_args	farm_id/house_id/auth_farm_id를 도구 호출용 기본 인자로 표준화 (권한 스코프 포함).
4	src/ai/	query_handler_simple.py	_call_llm_with_timeout	전통 Tool Use 루프를 asyncio.to_thread로 감싸 타임아웃/취소를 적용 (fallback 경로).
5	src/ai/	query_handler_simple.py	_unpack_llm_result	파이프라인 결과 dict를 (답변, 메시지, 도구사용, 응답타입, 출처) 튜플로 언팩.

L3. 대화 컨텍스트 구성 — 멀티턴 + VectorDB 검색

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	conversation_context.py	load_hybrid_context	session_id 최근 턴(PostgreSQL) + 과거 대화 벡터검색 결과를 하이브리드로 병합해 LLM 컨텍스트 구성.
2	src/ai/	conversation_context.py	_search_related_conversations	현재 질문을 임베딩 후 conversation_memory 컬렉션에서 유사 턴 N건 회수.
3	src/ai/	conversation_context.py	_classify_topic	질문의 도메인(농업/일반/인사)을 분류해 컨텍스트 선별 전략을 분기.
4	src/ai/	conversation_store.py	get_conversation_history	PostgreSQL에서 session_id 기반 최근 N턴 조회 (인메모리 폴백 포함).

L4. 3단계 분리형 파이프라인 실행

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/pipeline/	runner.py	run_3stage_pipeline_sync	L4의 총 진입점. analyze → collect → generate 3단계 순차 실행. 실패 시 전통 Tool Use 루프로 fallback.

L4-1. 질문 유형 분석 (Stage 1)				
#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/pipeline/	question_analyzer.py	analyze_question	LLM에게 {질문유형, 필요 도구 목록, 파라미터}를 JSON으로 생성 요청. greeting/conversation_ref는 2단계 스킵.
2	src/ai/pipeline/	question_analyzer.py	fast_classify	극히 명확한 인사/답답은 LLM 호출 없이 규칙으로 즉시 분류 (레이턴시 단축).
3	src/ai/pipeline/	question_analyzer.py	_parse_analysis_json / _validate_analysis	LLM 응답에서 JSON 블록 추출·스키마 검증 후 정규화.
4	src/ai/pipeline/	question_analyzer.py	_build_safe_fallback	분석 실패 시 기본 도구셋을 주입한 안전한 분석 결과를 생성.
5	src/ai/pipeline/	prompts.py	build_analyzer_system_prompt	Stage 1 전용 시스템 프롬프트(역할·출력 JSON 스키마·few-shot) 구성.

L4-2. 데이터 수집 + 충분성 검증 (Stage 2)				
#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/pipeline/	data_collector.py	DataCollector.collect	analysis_result.required_data 기반으로 도구 호출 태스크 목록을 구성·순차 실행·결과 누적.
2	src/ai/pipeline/	data_collector.py	DataCollector._expand_multi_house	multi_house=True인 도구 호출을 각 재배사별 개별 태스크로 팬-아웃 확장.
3	src/ai/pipeline/	data_collector.py	DataCollector._execute_single_tool	단일 도구를 tools_executor로 dispatch, 반환값·출처 URL을 표준 형식으로 수집.
4	src/ai/pipeline/	data_collector.py	DataCollector._collect_sources	웹 검색/URL fetch 결과에서 인용 가능한 출처 링크를 추출·중복 제거.
5	src/ai/pipeline/	validators.py	validate_with_llm	LLM에게 수집 데이터가 질문 답변에 충분한지 판정 요청. 부족 시 추가 도구 호출 지시.
6	src/ai/pipeline/	validators.py	summarize_collected_data	수집 결과를 LLM 검증용 요약 텍스트로 변환 (토큰 절약).

L4-3. 답변 생성 (Stage 3)				
#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/pipeline/	answer_generator.py	generate_answer	수집 데이터 + 분석 결과 + 대화 이력을 LLM에 투입해 최종 답변 생성. response_type/used_sources 포함해 반환.
2	src/ai/pipeline/	answer_generator.py	_format_collected_data	수집된 raw 결과를 LLM 입력용 마크다운 표·텍스트로 정형화.
3	src/ai/pipeline/	answer_generator.py	_extract_used_sources / _remove_source_tags	답변 본문에서 [SRC:...] 태그를 파싱해 출처 리스트로 분리, 본문에서 태그 제거.
4	src/ai/pipeline/	answer_generator.py	_generate_simple_response	인사/확인 등 도구 불필요한 질문은 도구 없이 간결 답변 생성.
5	src/ai/pipeline/	prompts.py	build_answer_system_prompt	Stage 3 전용 시스템 프롬프트(농장정보·화법·출처 태그 규칙) 구성.

L5. 도구(Tool) 실행 — tools_executor 라우터				
#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	tools_executor.py	execute_tool	도구명에 따라 tools_data/tools_control/tools_search/opinet_tools로 dispatch. 라우터 계층.
2	src/ai/	tools_definition.py	(module-level TOOLS)	Ollama Function Calling용 JSON 스키마. 도구 이름/파라미터/설명을 LLM에 노출.
3	src/ai/	tools_data.py	search_farm_knowledge	Chroma farm_knowledge 컬렉션에서 벡터 검색. 농장 권한·house_id·doc_type where 필터

구성.

4	src/ai/	tools_data.py	get_farm_realtime_data	PostgreSQL에서 재배사 센서/릴레이/생육 실시간 데이터 조회.
5	src/ai/	tools_control.py	control_relay / control_relays_batch	LLM 요청에 따라 릴레이 ON/OFF/반전을 DB에 반영. 권한·존재 검증 포함.
6	src/ai/	tools_search.py	search_web	웹 검색 엔트리. 엔진 호출→필터→재시도→VectorDB 캐싱까지 담당.
7	src/ai/	opinet_tools.py	get_oil_price / get_lowest_stations	Opinet API(실시간) 또는 PostgreSQL 풀백으로 유가 정보 반환.

L6. LLM 호출 — Ollama 통신

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	llm_client.py	get_llm_response_with_tools	전통 Tool Use 루프(fallback). 모델·메시지·도구 스키마를 Ollama에 전달, 반복적 tool_call 처리.
2	src/ai/	llm_client.py	_execute_and_merge_tools	LLM 응답의 tool_calls를 병렬 실행·결과를 message history에 병합.
3	src/ai/	llm_client.py	_normalize_tool_arguments	LLM이 반환한 tool args의 타입·한글 필드명·누락 파라미터를 표준화.
4	src/ai/	llm_client.py	_build_tool_detail_message / _report_progress	도구 실행 과정을 사용자용 진행 메시지로 가공해 progress_queue에 송출.
5	src/ai/	llm_transport.py	_ollama_chat	전송 계층 단일 엔트리. package → direct-HTTP → MCP 순으로 자동 시도·오류 시 다음 경로 폴백.
6	src/ai/	llm_transport.py	_pkg_ollama_chat / _direct_ollama_chat / _mcp_ollama_chat	Ollama Python 패키지 / 직접 HTTP / MCP 서버 — 3가지 전송 방식 구현.
7	src/ai/	llm_transport.py	_get_model_gpu_ratio / _get_model_ctx_options	설치 VRAM/모델 크기에 따라 num_gpu·num_ctx·num_predict 동적 산출.
8	src/ai/	llm_transport.py	_get_model_name	env MODEL_NAME 기반 실사용 모델 결정 (설치 유무 확인 후 폴백).
9	src/ai/	llm_transport.py	_log_llm_request_json / _log_llm_response_json	LLM 요청/응답을 JSON 라인으로 상세 로깅 (디버깅·성능 분석용).

L7. 응답 후처리 — 정제·검증·태깅

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	llm_response.py	clean_response / process_llm_answer	응답에서 think/thinking 블록 제거, 마크다운 표 정렬, 도구 결과 삽입 정제.
2	src/ai/	llm_response_processing.py	_check_answer_retry	응답이 지시 위반/빈 값이면 재시도 프롬프트를 구성해 LLM에 재호출.
3	src/ai/	llm_response_processing.py	_determine_response_type	tools_used 조합으로 response_type(control/info/greeting/error) 판정.
4	src/ai/	llm_response_processing.py	_build_structured_result	최종 dict(response_type, answer, sources, tools_used, ...) 조립.
5	src/ai/	llm_response_processing.py	_build_farm_info_text	현재 활성 농장/재배사 목록 텍스트를 구성해 프롬프트/응답에 주입.
6	src/ai/	llm_message_utils.py	extract_text_from_response / extract_tool_calls	Ollama package 객체와 dict 응답을 통일된 방식으로 파싱.
7	src/ai/	utils.py	strip_think_tags / detect_korean	think-tag 제거·한국어 여부 판정 (공통 정규식 유틸).

L8. 대화 턴 저장 (멀티턴 누적)

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	conversation_context.py	save_conversation_turn_hybrid	응답 완료 직후 PostgreSQL에 턴 저장(즉시) + VectorDB 백그라운드 Thread 저장.
2	src/ai/	conversation_context.py	_async_vectordb_save	daemon Thread로 질문+응답을 임베딩·conversation_memory 컬렉션에 업서트.
3	src/ai/	conversation_context.py	_prune_old_conversations	farm_id별 저장된 턴이 30건을 초과하면 가장 오래된 것부터 삭제.
4	src/ai/	conversation_store.py	save_conversation_turn	PostgreSQL conversation_turns 테이블에 session_id/질문/응답/메타 INSERT.
5	src/ai/	stats_collector.py	record_query	응답시간/도구사용/검색성공률을 인메모리 stats에 Thread-safe 누적.

L9. SSE 스트리밍 — 실시간 진행 상태 전송

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	llm_client.py	_report_progress	단계별 진행 메시지를 ThreadQueue에 put. API 계층이 consume.
2	api/	app.py	query_llm_stream (SSE loop)	progress_queue를 while True로 polling하여 `data: {json}␣␣␣` 형식으로 클라이언트 송출. 종료 sentinel 수신 시 break.
3	src/ai/	query_utils.py	split_for_streaming	긴 응답을 자연 경계(문장/마크다운)로 분할해 점진 송출 단위 구성.

L10. 파일 첨부 처리 (선택적 전처리)

#	디렉토리	파일명	함수명	함수기능역할
1	src/ai/	file_processor.py	process_uploaded_files	업로드된 파일 경로 리스트를 유형 감지 후 LLM 입력 텍스트로 변환.
2	src/ai/	file_processor.py	read_csv_file / read_excel_file / read_pdf_file / read_text_file	CSV/Excel/PDF/텍스트 개별 파서. 100MB/100행/1만자 등 한도 적용.